

An Agent Workspace as a Measuring Instrument for the OS It Lacks

Anonymous Author(s)

ABSTRACT

AI agents are becoming long-lived services that plan, invoke tools, and persist state, yet they are still hosted by application-layer workspaces built on operating-system abstractions designed for static workloads. We built and ran such a workspace—an open-source, multi-agent chat application with persistent per-agent memory—and report where its implementation hit walls that are missing OS abstractions. We enumerate six workarounds, measure all six (three scaling, three correctness cliffs), and map each onto an OS abstraction that would remove it. The contribution is consumer-side evidence, not a new OS, for the set of abstractions the agent era is starved for.

1 INTRODUCTION

Operating systems expose processes, threads, files, sockets, and resource controllers as the units of execution, composition, and accounting. These abstractions were designed for workloads that are static, predictable, and short-lived relative to human attention. AI agents invert all three properties: they are long-lived, semantically adaptive, and interact with tools and environments in ways the runtime cannot predict. A recent workshop series argues that this mismatch is not incidental but structural, and that operating systems themselves must become *agentic* [1].

Most of the evidence offered for this claim comes from the *producer* side of the OS interface: new kernel primitives, new schedulers, new isolation models. This paper offers evidence from the *consumer* side. We built and operate a real, open-source agent workspace—a multi-agent chat application in which a human routes work to named agents via channel mentions, and each agent retains memory across sessions. The application is small (a few hundred lines) and states its non-goals explicitly. We found that at every hard problem, the limiting factor was not the application’s logic but the absence of an operating-system abstraction that should plausibly exist.

This is a position paper, not a new system: the contribution is the mapping. Each of the workspace’s admitted workarounds corresponds to an OS-level abstraction that would dissolve it. We measure all six to show they are not cosmetic: three scaling walls (latency vs concurrency, tokens vs history, stored rows vs turns) and three correctness cliffs (isolation, delegation, capability routing), each failing on 100% of fault-injection trials. Each is technically re-implementable in user space, but only by re-building, per application, the scheduling and managed-state primitives that belong one layer down—and by getting their corner cases right every time.

2 THE WORKSPACE

The system is an open-source agent workspace (anonymized for review). A human types into a single chat channel; messages beginning with @Name are routed to the named agent. Each agent has a persona and a private, append-only memory. On each turn the agent’s full history is replayed into the prompt, the model is invoked through a standard OpenAI-compatible endpoint, and the reply is stored back. Memory is isolated per agent: a fact told to one agent is not visible to another.

The implementation comprises a Node.js HTTP router, a SQLite store for channel history and per-agent memory, and a minimal web frontend. It uses the host Node runtime’s built-in SQLite module; no external agent framework, vector database, or daemon is involved. The system is deployed for our own use.

What makes the system useful as evidence is not its capability but the explicitness of its non-goals. By design, it has:

- **No multi-agent concurrency.** Messages in a channel are processed through one serial queue. Two agents cannot act on the same state at the same time.
- **No memory compression.** The entire history is replayed into the prompt on every turn. There is no summarization or archival layer.
- **No inter-agent isolation.** All agents share one process and one database; there is no boundary preventing one agent from reading another’s state.
- **No capability routing.** A mention is parsed by a regular expression; there is no matching of work to the agent best suited for it, and no delegation between agents.

These are not oversights. Each is a deliberate non-goal that kept the implementation readable. We now argue that each is also a place where an operating-system abstraction is conspicuously absent.

3 WORKAROUNDS AND THE ABSTRACTIONS THEY WANT

Table 1 lists the six workarounds and the OS abstraction each implies. They split into three scaling-pathology walls (growth curves) and three correctness-cliff walls (invisible until the first adversarial event, surfaced via fault-injection rates). We measure all six below in the uncompressed configuration (the workspace ships a summarization switch that bounds prompt-token growth; we disable it to surface the walls). Single runs on a developer laptop, one OpenAI-compatible model (glm-5.1), reproducible from the artifact (anonymized for review).

3.1 Measured: the serial queue

3.2 Measured: six walls, two instruments

The six walls are measured with two instruments: a growth curve for the three scaling-pathology walls, and a fault-injection rate for

Table 1: Each workaround maps to an OS abstraction that would remove it.

Workaround (what is hand-rolled)	OS abstraction wanted
Serial queue (no concurrency)	Semantics-aware agent scheduling
Full-history replay (no compression)	Long-lived managed state
One process + one DB (no boundary)	Execute-only / attested isolation
JSON personas (hand-authored)	Skills as unit of composition
@Name regex (no matching)	Agent-facing interface / IPC
SQLite file (no managed object)	Agent state as OS object

Table 2: All six walls measured. Scaling walls: growth shape. Cliffs: fault-injection cross-rate.

Wall	Family	Result
Serial queue	scaling	latency 3.3→27.3 s over $N=1.8$ (linear)
Prompt replay	scaling	tokens 39→308 over 8 turns (linear)
Managed state	scaling	rows 2→40 over 20 turns (linear, no eviction)
Isolation	cliff	boundary crossed 100/100 trials
Composition	cliff	delegation reached target 0/20 trials
Routing	cliff	mismatched task refused 0/20 trials

the three correctness cliffs. Table 2 summarizes; the pattern is that every scaling wall grows linearly and every cliff fails on 100% of trials. Each wall is the absence of the abstraction Table 1 names: a degenerate scheduler (no global order basis), full-history replay (no compression), append-only storage (no eviction), an advisory boundary (no runtime mediation), no inter-agent call path (no composition), and @name regex (no capability matching). The application cannot fix any of them without re-implementing, in user space, the abstraction one layer down—and it cannot do isolation correctly at all, being both the enforcer and the enforced-upon (§4).

4 WHY THESE ARE ABSTRACTIONS, NOT FEATURES

A reviewer may object that the items in Table 1 are missing *features*, not missing *abstractions*. The distinction is the contribution, so we give a falsifiable test: *a requirement is an abstraction, not a feature, when any application-layer implementation of it is necessarily best-effort and cannot be verified correct in isolation*. A feature lives inside one program’s logic and can be made correct there; an abstraction is a contract that must be guaranteed by the layer beneath, or not at all.

Isolation is the clearest case (Table 2 measures it): any boundary an application draws is advisory—a bug, a misconfigured tool, or a prompt-injected agent steps over it, and the application has

no vantage point to detect the crossing. Only the runtime, mediating every memory access, can enforce one that holds. The same structure—enforcer and enforced-upon are the same program—repeats across the other rows: concurrency safety, context compression, capability routing each cannot be verified by the process whose own behavior it constrains.

This is also why the walls are not specific to our workspace. Any application that lets agents persist, collaborate, or be selected by capability occupies the same position—enforcer and enforced-upon at once—and hits the same walls, because the walls are in the contract between the application and its runtime, not in the application’s logic. The fact that a few-hundred-line application is forced to hand-roll all of these, badly, is the evidence.

5 RELATION TO THE AGENTICOS AGENDA

Each row aligns with a topic the AgenticOS community has identified from the producer side: agent-aware resource control [2], long-lived state for agent memory [1], execute-only and attested-channel isolation [4, 5], skills as composition unit [3], agent-facing interface redesign [6]. But the 1st-edition agenda is weighted roughly ten-to-two toward performance over security: the three scaling walls we measure sit on the well-served side, while the three correctness cliffs—*isolation*, *composition*, *routing*—occupy the under-served ground. The asymmetry in our evidence mirrors the asymmetry the field carries.

6 WHAT WE ARE NOT CLAIMING

We do not claim a new operating system. We do not claim that the abstractions named in Table 1 are the *correct* ones, only that they are *wanted*. We do not claim the workspace is representative of all agent applications; we claim that any application with its shape—persistent, multi-agent, capability-routed—will hit the same walls because the walls are in the runtime contract. We do not claim these abstractions are easy; we claim their absence is expensive, and we have the measurements to show it. The measurements are limited: single-run, one model, small N (8–20–100), no seeds or CIs. They show growth *shape* (three scaling walls) and cross-rates (three cliffs), not magnitudes. The skills-composition probe is a weak proxy (no scalar metric for composition exists); the routing probe uses an LLM judge.

7 CONCLUSION

An application-layer agent workspace that states its non-goals explicitly is a measuring instrument for the abstractions its operating system lacks. We built one, measured all six of its walls (three scaling, three cliffs), and mapped six to OS-level abstractions the agent era needs. Consumer-side evidence of this kind complements the producer-side primitives the community is beginning to propose, toward a contract agent applications request rather than re-implement.

REFERENCES

- [1] AgenticOS Workshop Series. *Operating Systems Design for AI Agents*. SOSP 2026 (2nd edition); ASPLOS 2026 (1st edition). <https://os-for-agent.github.io/>
- [2] Y. Zheng, J. Fan, Q. Fu, et al. *AgentCgroup: Understanding and Controlling OS Resources of AI Agents*. AgenticOS 2026.
- [3] L. Chen, Z. Wang, W. Zheng, et al. *Skills are the new Apps—Now It Is Time for Skill OS*. AgenticOS 2026.

- [4] R. Tiwari, D. Williams. *Execute-Only Agents: Architectural Defense Against Prompt Injection for AI Agents*. AgenticOS 2026.
- [5] Q. Wu, W. Zhang, G. Fang, et al. *Grimlock: Guarding High-Agency Systems with eBPF and Attested Channels*. AgenticOS 2026.
- [6] Y. Wang et al. *Rethinking OS Interfaces for LLM Agents*. AgenticOS 2026.